C# Session 7: Detailed Explanations and Examples

In this session, we will cover several important concepts in C#, including generic classes, generic methods, generic constraints, collections (both generic and non-generic), examples based on `ICollection`, `IList`, and `IDictionary`, iterating collections using `foreach`, and using tuples to pass multiple values to a function. Each topic will be explained in detail with examples to enhance your understanding.

---

1. C# Generic Classes

Overview of Generics in C#

- **Generics** allow you to define classes, methods, and interfaces with a placeholder for the data type, enabling code reusability and type safety.
- Introduced in C# 2.0, generics are widely used in the .NET framework, especially in collections.

Key Benefits of Using Generics

- **Type Safety**: Ensures that the data types used in collections are checked at compile time, reducing runtime errors.
- **Code Reusability**: Write a single class or method that can work with any data type.
- **Performance**: Reduces the need for boxing and unboxing, which can improve performance.

Example of a Generic Class

```
csharp
VerifyOpen In EditorRunCopy code
1using System;
2
3public class DataStore<T>
4{
5    public T Data { get; set; }
6
7    public DataStore(T data)
8    {
9        Data = data;
10    }
11
12    public void DisplayData()
13    {
14        Console.WriteLine($"Data: {Data}");
15    }
16}
17
18class Program
19{
20    static void Main()
21    {
22        DataStore<string> stringStore = new DataStore<string>("Hello,
Generics!");
23        stringStore.DisplayData(); // Output: Data: Hello, Generics!
24
25        DataStore<int> intStore = new DataStore<int>(42);
26        intStore.DisplayData(); // Output: Data: 42
27    }
28}
```

---

2. C# Generic Methods

Definition

- Generic methods allow you to define a method with a type parameter, enabling it to operate on different data types.

Example of a Generic Method

```
csharp
VerifyOpen In EditorRunCopy code
1using System;
2
3public class GenericMethods
4{
5    public void Print<T>(T value)
6    {
7        Console.WriteLine(value);
8    }
9}
10
11class Program
12{
13    static void Main()
14    {
15        GenericMethods gm = new GenericMethods();
16        gm.Print(100); // Output: 100
17        gm.Print("Hello, World!"); // Output: Hello, World!
18    }
19}
```

---

3. C# Generic Constraints

Definition

- Generic constraints allow you to specify the requirements for the types that can be used with a generic class, method, or interface.

Types of Constraints

1. **Value Type Constraint (`where T : struct`)**
2. **Reference Type Constraint (`where T : class`)**
3. **Default Constructor Constraint (`where T : new()`)**
4. **Interface Constraint (`where T : IMyInterface`)**
5. **Base Class Constraint (`where T : BaseClass`)**
6. **Multiple Constraints**

Example of Generic Constraints

```
csharp
VerifyOpen In EditorRunCopy code
1public class GenericWithDefaultConstructor<T> where T : new()
2{
3    public T CreateInstance()
4    {
```

```
5        return new T();
6    }
7}
8
9// Usage
10public class MyClass
11{
12    public string Name { get; set; } = "Default";
13}
14
15var instanceCreator = new GenericWithDefaultConstructor<MyClass>();
16var myClassInstance = instanceCreator.CreateInstance();
17Console.WriteLine($"Name: {myClassInstance.Name}"); // Output: Name: Default
```

---

4. C# Collections: Generic vs Non-Generic

Generic Collections

- Defined in `System.Collections.Generic`, they provide type safety and better performance.
- Common types: `List<T>`, `Dictionary<TKey, TValue>`, `Queue<T>`, `Stack<T>`.

Example of Generic List

```
csharp
VerifyOpen In EditorRunCopy code
1using System;
2using System.Collections.Generic;
3
4class Program
5{
6    static void Main()
7    {
8        List<int> numbers = new List<int> { 1, 2, 3 };
9        foreach (var number in numbers)
10        {
11            Console.WriteLine(number); // Output: 1 2 3
12        }
13    }
14}
```

Non-Generic Collections

- Defined in `System.Collections`, they store elements as `object` types, lacking type safety.
- Common types: `ArrayList`, `Hashtable`.

Example of Non-Generic ArrayList

```
csharp
VerifyOpen In EditorRunCopy code
1using System;
2using System.Collections;
3
4class Program
5{
6    static void Main()
7    {
8        ArrayList arrayList = new ArrayList { "Hello", 123, DateTime.Now };
```

```
9        foreach (object item in arrayList)
10        {
11            Console.WriteLine(item);
12        }
13    }
14}
```

---

5. Iterating Collections Using `foreach`

Definition

- The `foreach` loop is a convenient way to iterate through collections such as arrays, lists, and dictionaries.

Example of Iterating Through an Array

```
csharp
VerifyOpen In EditorRunCopy code
1int[] numbers = { 1, 2, 3, 4, 5 };
2foreach (int number in numbers)
3{
4    Console.WriteLine(number); // Output: 1 2 3 4 5
5}
```

Example of Iterating Through a Dictionary

```
csharp
VerifyOpen In EditorRunCopy code
1Dictionary<string, int> ages = new Dictionary<string, int>
2{
3    { "Alice", 30 },
4    { "Bob", 25 }
5};
6
7foreach (var entry in ages)
8{
9    Console.WriteLine($"{entry.Key}: {entry.Value}"); // Output: Alice: 30,
Bob: 25
10}
```

---

6. Using Tuples to Pass Multiple Values to a Function

Definition

- Tuples are a lightweight way to group multiple values into a single object.

Example of Using Tuples

```
csharp
VerifyOpen In EditorRunCopy code
1public (int, string) GetPerson()
2{
3    return (1, "Alice");
4}
5
6class Program
7{
```

```
8    static void Main()
9    {
10        var person = GetPerson();
11        Console.WriteLine($"ID: {person.Item1}, Name: {person.Item2}"); //
Output: ID: 1, Name: Alice
12    }
13}
```

Conclusion

This session covered essential concepts in C# related to generics, collections, and iteration. Understanding these topics will enhance your ability to write flexible, reusable, and type-safe code. Each example provided illustrates practical applications of these concepts, making it easier to grasp their importance in C# programming.